

Documentación de la PEC 4

Índice

1. Enunciado de la práctica.
2. Descripción de la solución adoptada. En lenguaje natural explicará brevemente cómo se va a resolver el problema y cómo se va a organizar la solución.
3. Diagrama explicativo de los diferentes módulos que participan en la aplicación.
4. Descripción en cada módulo de los elementos que contienen (Datos y subprogramas) y descripción del tipo de datos principal de la aplicación.

Enunciado de la práctica

Realizar un programa en C+/- para Gestionar las Reservas de los Apartamentos de un Edificio (GesRAE). En cada edificio, los apartamentos pueden ser catalogados según sus prestaciones como Básico, Normal o de Lujo. Todos los apartamentos con las mismas prestaciones son equivalentes y será el programa el encargado de realizar la asignación concreta al realizar la reserva.

El sistema GesRAE deberá tener las siguientes capacidades mínimas:

- Hasta 5 edificios.
- Hasta 20 Apartamentos por edificio.

Las operaciones del programa serán las siguientes:

- Editar Edificio.
- Listar Edificios.
- Apartamentos Disponibles.
- Reservar Apartamento.
- Reservas Mensuales de Apartamento.

Texto copiado del documento de enunciado, se puede encontrar [aquí](#).

Descripción de la solución adoptada. En lenguaje natural explicará brevemente cómo se va a resolver el problema y cómo se va a organizar la solución

El programa sigue la metodología de Tipo Abstracto de Datos, por ello, se pueden diferenciar 3 grandes partes del programa: manejo de la interfaz, manejo de las

respuestas, y manejo de las fechas. Mi forma de programarlo consistía en: uno, declarar los límites del programa (número máximo de apartamentos, por ejemplo); dos, desarrollar las estructuras principales (como el `TipoEdificio`); tres, desarrollar el bucle principal del programa y manejar el menú; y, cuatro, desarrollar cada pregunta, empezando por la primera y acabando por la última (cabe aclarar que aquí hubo una excepción a la hora de revisar las reservas, primero era necesario declarar cómo se iban a guardar).

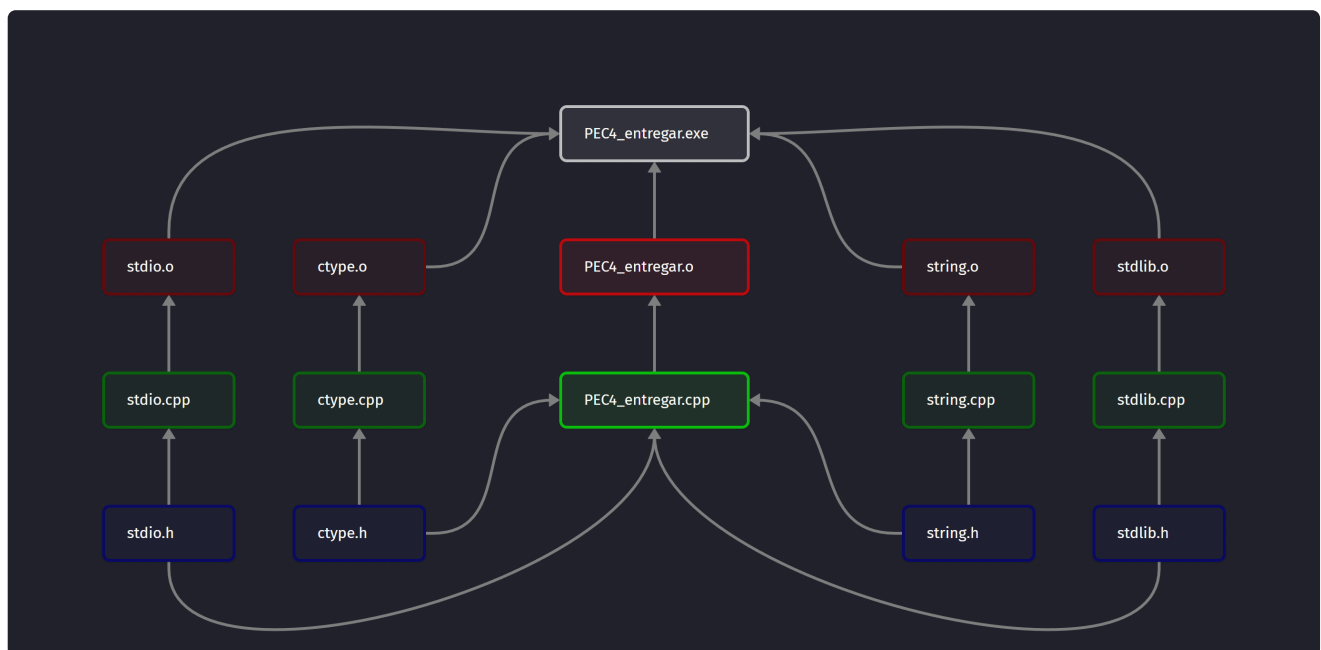
Diagrama explicativo de los diferentes módulos que participan en la aplicación

Debido a que, pese a conocer bien la metodología de desarrollo de módulos y la creación de proyectos, no he entendido como compilarlo todo junto en un proyecto dentro del editor de Code::Blocks UNED-ISSI 2009, he optado por no crear mis módulos personalizados.

Los módulos usados son:

- *Standard Input-Output Header* (`stdio.h`): Para poder imprimir cosas por consola.
- *Charter Type Header* (`ctype.h`): Para poder transformar las entradas del usuario a mayúsculas, para así solo manejar las mayúsculas, esto añade robustez.
- *String Header* (`string.h`): Para poder guardar el nombre (*string*) que se de al edificio.
- *Standard Library Header* (`stdlib.h`): Para poder acceder a la palabra reservada `NULL` de forma correcta.

Como todos los módulos estándar apuntan al mismo fichero, el diagrama es bastante sencillo. [Diagrama de módulos](#):



Descripción en cada módulo de los elementos que contienen (Datos y subprogramas) y descripción del tipo de datos principal de la aplicación

Me he tomado la licencia de juntar ambos puntos, de esta forma, esta documentación guarda más cohesión con el programa, pues se sigue un orden específico y se explican tipos de datos que se usan en los subprogramas y estructuras. A continuación, desarrollo las partes en función del orden en el que aparecen en el programa:

Tipos de datos

- `typedef char TipoNombre[21];` : El nombre que puede tener un edificio, es 20 y no 21 por el carácter que indica el final de la secuencia de caracteres ("`\n`").
- `typedef char TipoReferencia[6];` : La referencia que se pasa, por ejemplo "APT01B02", he decidido recoger solo el "01B02" e incluir el "APT" como parte de la impresión del programa, esto ahorra un mínimo de espacio en el programa.
- `typedef int TipoIdentificador;` : El identificador de los edificios.
- `typedef int TipoNumerosApartamentos[MAX_Apartamentos];` : Para saber las habitaciones exactas que se han reservado, para no volver a reservarlas, por eso, se guardan aquí. Como solo hay 20 apartamentos máximos de un tipo (o los que se especifiquen), el límite nunca puede ser superado, pues se comprueba si un elemento se va a repetir en el vector antes de añadirlo.
- `typedef enum TipoMes { Enero, Febrero, Marzo, Abril, Mayo, Junio, Julio, Agosto, Septiembre, Octubre, Noviembre, Diciembre };` : Sencillamente los meses que existen. Se declaran de forma enumerada. Además, más tarde se usará para la creación de la fecha, esto se aplica para las siguientes 2 tipos definidos.
- `typedef int TipoAnno;` : Como es un dato especial (con sus normas, sus construcciones y sus métodos de comprobación), he querido meterlo en un tipo de dato.
- `typedef int TipoDia;` : Lo mismo que en el `TipoAnno`, cuenta con sus normas, construcciones y funciones, métodos de comprobación, por eso lo he metido dentro de un tipo de dato.

Estructuras y sus subprogramas

- `TipoInterfaz` : Esta es una práctica común en mi metodología propia de desarrollo de programas. Suelo dedicar una estructura conformada por un conjunto de funciones y procedimientos que se encargan única y exclusivamente del manejo de la interfaz. En este caso, de espaciar y de imprimir el menú de inicio.
 - Subprogramas:

- `void Espaciar(int numero)` se encarga de imprimir un número de espacios, según se le especifique.
- `char ImprimirMenuInicio()` se encarga de imprimir el menú principal (que se puede consultar en la segunda página del [enunciado del PDF](#)) y obtener la respuesta del usuario, luego la retorna.
- **TipoFecha** : Aquí se coleccionan todo el conjunto de procedimientos, funciones y tipos que son necesarios para trabajar con las fechas.
 - Datos:
 - `TipoDia dia` : Toda fecha tiene su día.
 - `TipoMes mes` : Toda fecha tiene su mes.
 - `TipoAnno anno` : Toda fecha tiene su año.
 - Subprogramas:
 - `bool ComprobarDia(TipoFecha fecha)` : Comprueba que el día esté correcto. Se pide una fecha porque es necesario el día, el mes (pues el rango de días posibles cambia con este), y el año (pues el rango de días del mes de febrero puede cambiar según el año).
 - `bool ComprobarMes(TipoMes mes)` : Comprueba que el mes sea correcto.
 - `bool ComprobarAnno(TipoAnno anno)` : Comprueba que el año sea correcto.
 - `bool ComprobarBisiesto` : Comprueba si el año es bisiesto o no. Todas las comprobaciones retornan `true` si es correcto y `false` si no.
 - `int ObtenerLongitudMes(TipoMes mes, TipoAnno anno)` : Obtiene y retorna cuántos días tiene un mes, pide el mes (obviamente) y el año, pues el mes de febrero puede variar en días en función del año.
 - `int ObtenerLongitudAnno(TipoAnno anno)` : Obtiene la cantidad de días que tiene un año.
 - `int ObtenerDias(TipoFecha fecha)` : A partir de una fecha, devuelve la cantidad de días que han pasado desde el 1 de enero de 1601. He tomado esta referencia porque se pidió la reutilización de código, y esa fue la referencia que se especificó en la PEC 3, además, resulta bastante útil que el 1 de enero de 1601 fuese lunes.
 - `TipoFecha ObtenerFecha(int dias)` : Obtiene y devuelve la fecha a partir de los días que han pasado desde el 1 de enero de 1601. Se podría decir que es como la función anterior, pero al revés.
- **TipoDiasReservados** : Esta es la lista enlazada que contiene los datos que se van a guardar.
 - Datos:
 - `int N_Habitacion` : El número de habitación (el "N_" siempre hace referencia a "Número").

- `int calidadHabitacion` : Guardado como 0, 1 o 2, en función de si el apartamento es Básico, Normal o Lujoso, respectivamente.
- `int N_deReserva` : El número de reserva del año, o, en otras palabras, la cantidad de reservas que se hayan en ese año. Empieza en 0, aunque siempre se imprime por encima de 0.
- `TipoFecha fecha` : Guarda la fecha de la reserva (la reserva ocupa varios días, todos se guardan, pero uno por cada espacio de la lista enlazada).
- `TipoDiasReservados* siguiente` : Como toda lista enlazada, debe de apuntar a otra estructura para seguir avanzando.
- Extra: Fuera de la zona de declaración, se `TipoDiasReservados` , se escribe `typedef TipoDiasReservados* TipoPuntero;` , esto se hace para poder acceder a las reservas.
- `TipoEdificio` : Aquí se definen los datos que van a tener los edificios.
 - Datos:
 - `TipoNombre nombre` : Todo edificio tiene su nombre.
 - `TipoDiasReservados* informacionReservado` : Todo edificio cuenta con su reserva.
 - `int N_apartamentoBasico, N_apartamentoNormal, N_apartamentoLujo` : Todo edificio cuenta con su número de apartamentos básicos, normales o de lujo.
 - `bool baja, iniciado` : Todo edificio puede estar de baja, o sencillamente no iniciado (es decir, no se ha declarado un identificador para el edificio, entonces, este no debe de estar iniciado).
 - `TipoConjuntoEdificios` : Una de mis soluciones más creativas, como la cantidad máxima de edificios es variable y no quería nombrar edificios como variables de `TipoEdificio` ("edificio1", "edificio2"...), decidí crear un vector, donde el selector fuese el identificador. Cada edificio cuenta con sus métodos, que operan desde fuera de este en un conjunto de edificios.
 - Datos:
 - `TipoEdificio edificios[MAX_Edificios]` : Cada conjunto de edificios tiene sus edificios, en este caso, `MAX_Edificios` es constante, y refiere a una máxima (siempre que aparezca nombrada una variable como "`MAX_`", estaré nombrando una cantidad **máxima** de algo), en este, caso, 5 es el máximo.
 - Subprogramas:
 - `bool ComprobarIdentificador(TipoIdentificador identificador)` : Se comprueba que el identificador sea correcto, que no esté por debajo de 1 ni por encima del máximo.
 - `TipoEdificio ComprobarDisponibilidad(TipoEdificio edificio, TipoFecha fecha)` : Comprueba la disponibilidad de un edificio para una fecha concreta, devuelve un edificio con el número de

apartamentos de cada tipo con un valor correspondiente a cuántos apartamentos hay libres.

- Para los procedimientos posteriores de este apartado, es necesario tener en cuenta de que cada uno se corresponde a cada acción especificada en el enunciado.
 - `void EditarEdificio(TipoConjuntoEdificios &edificio)` : Edita un edificio, lo puede iniciar, dar de baja (para no volver a usarse hasta que se vuelva a editar), o cambia algún valor.
 - `void ListarEdificios(TipoConjuntoEdificios edificios)` : Imprime los edificios que están disponibles (iniciados y no de baja), con sus respectivos apartamentos.
 - `void ConsultarReservas(TipoConjuntoEdificios edificio)` : Muestra las reservas que hay para un edificio y fechas concretas. Las fechas (o fecha y días posteriores), se crean y se especifican dentro del procedimiento.
 - `void HacerReserva(TipoConjuntoEdificios &edificio)` : Reserva unos días dentro de un edificio, si hay disponibilidad, claro. Para eso se usa el `TipoNumerosApartamentos` de antes.
- `CalendarioMes` : Es la única estructura que no empieza su nombre por "Tipo" pues así se especificó en el enunciado: "se redefinirá como un TAD "CalendarioMes", incorporando las operaciones nuevas para cumplir las necesidades de esta cuarta práctica". Puede resultar un poco anticlimático, pero la función de consultar las reservas mensuales del apartamento con formato de calendario se encuentra dentro de esta estructura. La razón de esto es porque considero que dicho procedimiento está más cerca de la impresión de un calendario que del manejo de reservas. Por otro lado, cabe recalcar que la PEC 3 estaba aún más orientada al TAD, por lo que todo estaba dentro de una estructura.
 - Subprogramas:
 - `void ImprimirFecha(TipoMes mes, TipoAnno anno)` : Imprime la primera cabecera del calendario ("Enero", "Febrero"..., "2024", "2025"...).
 - `void ImprimirSeparador(int numero, char caracter)` : Similar a `Espaciar`, pero en este procedimiento es posible establecer el carácter a imprimir. He querido conservarlo pues se especifica que hay que reutilizar código. Es útil para imprimir cosas como la sucesión de guiones que se especifica en el enunciado que debe de llevar el calendario.
 - `void ImprimirSiglasSemana()` : Tan sencillo como imprimir la segunda cabecera del calendario, en este caso, es estática ("L M X...").
 - `void GenerarCalendario(TipoConjuntoEdificios edificio)` : El procedimiento más grande de la estructura. Se encarga de imprimir el calendario y generar un resumen de las fechas que interfieren con el mes seleccionado. Por ello, pide una fecha.